

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico Anual Lineamientos sobre informes



27 de diciembre de 2024

Castaño Mateo
110114
Rodrigo Pesa Leandro
110076

Oviedo Ignacio Sebastián
109821
Trezeguet Santiago Bautista
110343
Suxo Juan
108280

Primera parte: Introducción y primeros años

1. Algoritmo de victoria versión greedy

En esta sección del informe, se analizará un algoritmo greedy donde Sofía siempre resulta ganadora del juego.

1.1. Algoritmo

El funcionamiento del algoritmo es el siguiente: en cada turno, cuando Sofía debe elegir entre los valores disponibles en los extremos (derecha e izquierda), selecciona el valor más alto. Por otro lado, durante el turno de Mateo, Sofía toma la decisión por él y elige el valor más bajo.

A continuación se muestra el código de solución greedy al problema

```
1 def juego_greedy(vec):
2
3     v_sofia = [] * (len(vec)//2)
4     v_mateo = [] * (len(vec)//2)
5
6     for i in range(0, len(vec)):
7         if (i%2==0 or i==0):
8             resultado, vec = turno_sofia(vec)
9             v_sofia.append(resultado)
10        else:
11            resultado, vec = turno_mateo(vec)
12            v_mateo.append(resultado)
13    return sum(v_sofia)
```

Notese que, en cada iteración, se aplica una regla sencilla: elegir el valor mayor o menor, dependiendo de quién tenga el turno. Esta decisión se basa en seleccionar la mejor opción posible en el momento específico de la elección. Las funciones *turnosofia* y *turnomateo* son las encargadas de tomar esta decisión, evaluando el vector y comparando los valores en las posiciones inicial y final.

1.2. Complejidad

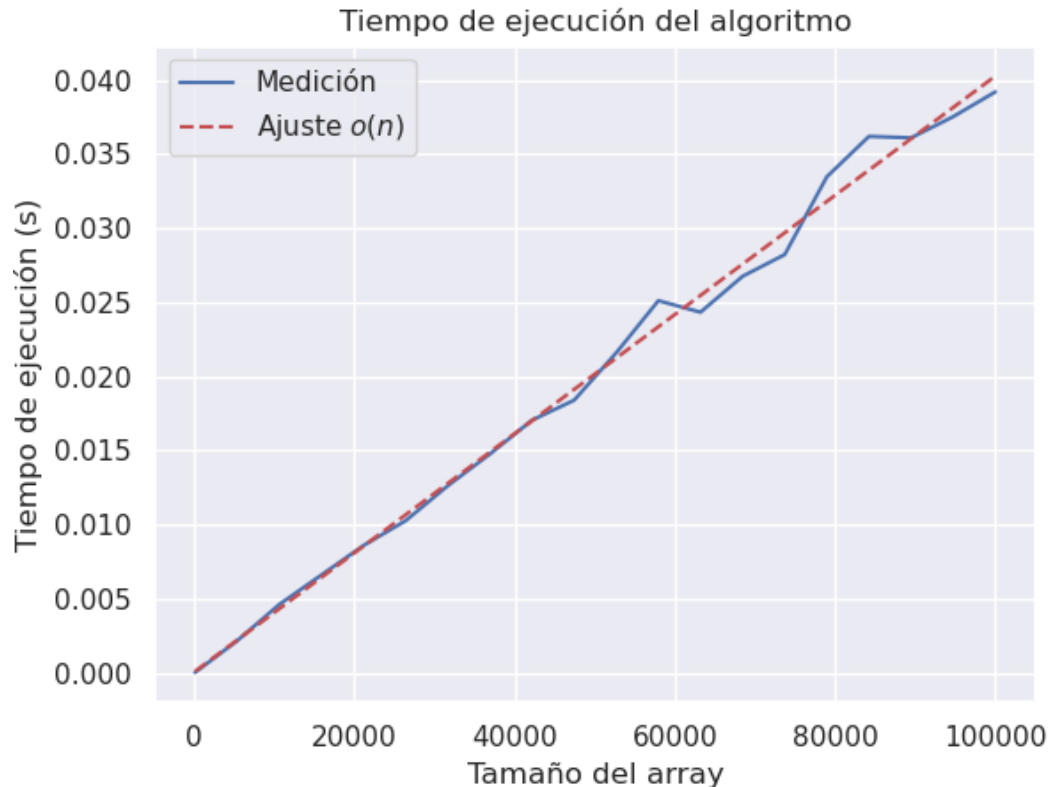
Para determinar la complejidad planteamos la ecuación de recurrencia. Notese que lo que está fuera del bucle es $O(1)$. El bucle recorre desde 0 hasta $n-1$, siendo n la longitud del vector. Dentro del bucle se realizan comparaciones cuya complejidad es $O(1)$. Por lo tanto, podemos afirmar que la ecuación de recurrencia es la siguiente:

$$T(n) = T(n-1) + n^0.$$

Por el teorema maestro, podemos concluir que la complejidad es $O(n)$. Esto se ve fácilmente ya que el bucle se repite $n-1$ veces y dentro del bucle la complejidad de las comparaciones es $O(1)$.

1.3. Mediciones

Para comprobar que la complejidad es la correcta, se probó utilizando la técnica de cuadrados mínimos, ajustando la función a una lineal. En la siguiente imagen podemos ver ese ajuste:



Como podemos ver, las mediciones tomadas tienden a ser una recta, por lo que podemos asumir que la complejidad teórica del algoritmo es correcta.

1.4. Optimalidad

El algoritmo garantiza una solución óptima para cualquier conjunto de valores. Para ilustrarlo, consideremos un ejemplo trivial con tres valores: $[9, 10, 2]$. En este caso, en el primer paso elegiríamos el valor más grande entre los dos extremos, es decir, el 9. Luego, seleccionaríamos el valor más pequeño de los restantes, que sería el 2. Finalmente, elegiríamos el valor 10, obteniendo un total de 19 puntos para nosotros y 2 puntos para nuestro hermano.

Es importante notar que, independientemente del orden de los valores, el resultado será siempre el mismo cuando el array es de tres.

Analicemos ahora un caso con cuatro valores: $[1, 9, 10, 2]$. En el primer paso, elegiríamos el 2 (el mayor de los extremos). Luego, tomaríamos el 1 (el menor de los valores restantes). A continuación, seleccionaríamos el 10 y finalmente el 9. Nuestro puntaje final sería 12, mientras que el de nuestro hermano sería 10. Como se observa, seguimos ganando.

Sin embargo, en este caso, aunque el orden del arreglo no afecta al ganador, sí influye en los resultados finales. Por ejemplo, si el arreglo fuera $[9, 1, 10, 2]$, nuestro puntaje sería 19 y el de nuestro hermano 3.

Podemos concluir que no siempre obtendremos la puntuación más alta posible. Sin embargo, para demostrar que siempre ganaremos, consideremos el siguiente arreglo: $[x_1, p, y_1]$, donde p representa un subarreglo de $n - m$ elementos, siendo n la longitud total del arreglo y m la suma de las monedas ya elegidas y las que están por ser seleccionadas.

Supongamos que x_1 es menor que y_1 . En ese caso, tras nuestra primera elección, obtendremos un puntaje de y_1 , y el arreglo quedará como $[x_1, p, y_2]$. Posteriormente, si x_1 es menor que y_2 ,

escogeremos x_1 para nuestro hermano. En este punto, nuestro puntaje total será y_1 , mientras que el de nuestro hermano será x_1 .

En la siguiente iteración, elegiremos el valor más alto de los extremos disponibles, lo que aumentará nuestro puntaje, asegurando que será mayor al de nuestro hermano, ya que cualquier número que seleccionemos será mayor que x_1 .

Si continuamos aplicando esta estrategia, llegaremos a dos escenarios posibles:

1. Nuestro turno ocurre cuando quedan tres monedas.
2. Nuestro turno ocurre cuando quedan cuatro monedas.

En ambos casos, como hemos demostrado previamente, siempre obtendremos la victoria.

Por lo tanto, podemos concluir que este enfoque greedy garantiza la victoria, aunque no necesariamente la puntuación más alta posible.

1.5. Conclusion

Podemos concluir que el algoritmo planteado resuelve nuestro problema de forma óptima, es decir, siempre garantiza nuestra victoria. Sin embargo, dentro de esa optimalidad, los resultados finales pueden variar dependiendo del orden de las monedas en el arreglo.

Con respecto a la complejidad, podemos decir que se comprobó tanto teóricamente como empíricamente que es $O(n)$.

Segunda parte: Mateo empieza a Jugar

2. Algoritmo de victoria versión programación dinámica

En esta sección del informe, se analizará un algoritmo de programación dinámica donde Sofía siempre resulta ganadora del juego con el mayor puntaje posible, dado que Mateo siempre elija el valor mas alto de los extremos.

2.1. Algoritmo

Definiendo:

$dp[i][j]$: el valor máximo que Sophia puede obtener si el rango de monedas considerado está entre las posiciones i y j . (inclusive).

Caso base:

Si $i = j$ (solo hay una moneda disponible):

$$dp[i][j] = \text{monedas}[i]$$

Casos generales:

Cuando hay más de una moneda, Sophia tiene dos opciones:

1. **Elegir la moneda al inicio (i):** La elección de Mateo (siguiendo una estrategia Greedy) quedará entre $\text{monedas}[i + 1]$ y $\text{monedas}[j]$:

- Si el mayor es $\text{monedas}[i + 1]$, Mateo elegirá esta, y el rango restante quedaría en $dp[i + 2][j]$ para Sophia.
- Si el mayor es $\text{monedas}[j]$, Mateo elegirá esta, y el rango restante quedaría en $dp[i + 1][j - 1]$ para Sophia.

De esta forma, el valor acumulado al elegir i será:

$$\text{pick}_i = \text{monedas}[i] + \begin{cases} dp[i + 2][j], & \text{si } \text{monedas}[i + 1] \geq \text{monedas}[j] \\ dp[i + 1][j - 1], & \text{si } \text{monedas}[i + 1] \leq \text{monedas}[j] \end{cases}$$

2. **Elegir la moneda al final (j):** La elección de Mateo (siguiendo una estrategia Greedy) quedará entre $\text{monedas}[i]$ y $\text{monedas}[j - 1]$:

- Si el mayor es $\text{monedas}[i]$, Mateo elegirá esta, y el rango restante quedaría en $dp[i + 1][j - 1]$ para Sophia.
- Si el mayor es $\text{monedas}[j - 1]$, Mateo elegirá esta, y el rango restante quedaría en $dp[i][j - 2]$ para Sophia.

De esta forma, el valor acumulado al elegir j será:

$$\text{pick}_j = \text{monedas}[j] + \begin{cases} dp[i + 1][j - 1], & \text{si } \text{monedas}[i] \geq \text{monedas}[j - 1] \\ dp[i][j - 2], & \text{si } \text{monedas}[i] \leq \text{monedas}[j - 1] \end{cases}$$

Maximización: Sophia seleccionará la opción que maximice su valor:

$$dp[i][j] = \max(\text{pick}_i, \text{pick}_j)$$

A continuación el algoritmo que implementa esta lógica:

```
1 def juego_programacion_dinamica(monedas):
2     n = len(monedas)
3     dp = [[0] * n for _ in range(n)]
4
5     for length in range(1, n + 1):
6         for i in range(n - length + 1):
7             j = i + length - 1
8             if i == j:
9                 dp[i][j] = monedas[i]
10            else:
11                # Si Sophia elige la moneda en la posición i
12                if monedas[i + 1] >= monedas[j]:
13                    pick_i = monedas[i] + dp[i + 2][j] if i + 2 <= j else monedas[i]
14                ]
15                else:
16                    pick_i = monedas[i] + dp[i + 1][j - 1] if i + 1 <= j - 1 else
17                    monedas[i]
18                # Si Sophia elige la moneda en la posición j
19                if monedas[i] >= monedas[j - 1]:
20                    pick_j = monedas[j] + dp[i + 1][j - 1] if i + 1 <= j - 1 else
21                    monedas[j]
22                else:
23                    pick_j = monedas[j] + dp[i][j - 2] if i <= j - 2 else monedas[j]
24                ]
25
26            dp[i][j] = max(pick_i, pick_j)
27
28    return dp[0][n - 1]
```

La complejidad temporal del algoritmo es $O(n^2)$, ya que se llena una tabla de tamaño $n \times n$, donde cada celda se calcula en tiempo constante $O(1)$. El proceso considera n posibles tamaños de subrangos, y para cada tamaño se analizan hasta n posiciones iniciales. En consecuencia, la combinación de estos factores conduce a un costo computacional cuadrático respecto al tamaño de entrada.

2.2. Seguimiento

Para validar el funcionamiento del algoritmo, se realizó un seguimiento paso a paso utilizando el conjunto de datos siguiente:

`monedas = [8, 15, 7, 9, 20, 1, 6]`

Casos base:

$$\begin{aligned} dp[0][0] &= 8 \\ dp[1][1] &= 15 \\ dp[2][2] &= 7 \\ dp[3][3] &= 9 \\ dp[4][4] &= 20 \\ dp[5][5] &= 1 \\ dp[6][6] &= 6 \end{aligned}$$

Tabla Inicial:

$$\begin{bmatrix} 8 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 15 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 7 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 20 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 6 \end{bmatrix}$$

Sub-secuencias de longitud 2:

$$\begin{aligned} dp[0][1] &= \text{máx}(\text{pick}_i = 8, \text{pick}_j = 15) = 15 \\ dp[1][2] &= \text{máx}(\text{pick}_i = 15, \text{pick}_j = 7) = 15 \\ dp[2][3] &= \text{máx}(\text{pick}_i = 7, \text{pick}_j = 9) = 9 \\ dp[3][4] &= \text{máx}(\text{pick}_i = 9, \text{pick}_j = 20) = 20 \\ dp[4][5] &= \text{máx}(\text{pick}_i = 20, \text{pick}_j = 1) = 20 \\ dp[5][6] &= \text{máx}(\text{pick}_i = 1, \text{pick}_j = 6) = 6 \end{aligned}$$

Sub-secuencias de longitud 3:

$$\begin{aligned} dp[0][2] &= \text{máx}(\text{pick}_i = 8 + 7, \text{pick}_j = 7 + 8) = 15 \\ dp[1][3] &= \text{máx}(\text{pick}_i = 15 + 7, \text{pick}_j = 9 + 7) = 22 \\ dp[2][4] &= \text{máx}(\text{pick}_i = 7 + 9, \text{pick}_j = 20 + 7) = 27 \\ dp[3][5] &= \text{máx}(\text{pick}_i = 9 + 1, \text{pick}_j = 1 + 9) = 10 \\ dp[4][6] &= \text{máx}(\text{pick}_i = 20 + 1, \text{pick}_j = 6 + 1) = 21 \end{aligned}$$

Sub-secuencias de longitud 4:

$$\begin{aligned} dp[0][3] &= \text{máx}(\text{pick}_i = 8 + 9, \text{pick}_j = 9 + 15) = 24 \\ dp[1][4] &= \text{máx}(\text{pick}_i = 15 + 9, \text{pick}_j = 20 + 9) = 29 \\ dp[2][5] &= \text{máx}(\text{pick}_i = 7 + 20, \text{pick}_j = 1 + 9) = 27 \\ dp[3][6] &= \text{máx}(\text{pick}_i = 9 + 6, \text{pick}_j = 6 + 20) = 26 \end{aligned}$$

Sub-secuencias de longitud 5:

$$\begin{aligned} dp[0][4] &= \text{máx}(\text{pick}_i = 8 + 22, \text{pick}_j = 20 + 15) = 35 \\ dp[1][5] &= \text{máx}(\text{pick}_i = 15 + 10, \text{pick}_j = 1 + 22) = 25 \\ dp[2][6] &= \text{máx}(\text{pick}_i = 7 + 21, \text{pick}_j = 6 + 10) = 28 \end{aligned}$$

Sub-secuencias de longitud 6:

$$\begin{aligned} dp[0][5] &= \text{máx}(\text{pick}_i = 8 + 27, \text{pick}_j = 1 + 24) = 35 \\ dp[1][6] &= \text{máx}(\text{pick}_i = 15 + 26, \text{pick}_j = 6 + 27) = 41 \end{aligned}$$

Sub-secuencia de longitud 7 (completa):

$$dp[0][6] = \text{máx}(\text{pick}_i = 8 + 28, \text{pick}_j = 6 + 25) = 36$$

Tabla Final:

8	15	15	24	35	35	36
0	15	15	22	29	25	41
0	0	7	9	27	27	28
0	0	0	9	20	10	26
0	0	0	0	20	20	21
0	0	0	0	0	1	6
0	0	0	0	0	0	6

Acumulación total: 36

2.3. Reconstrucción

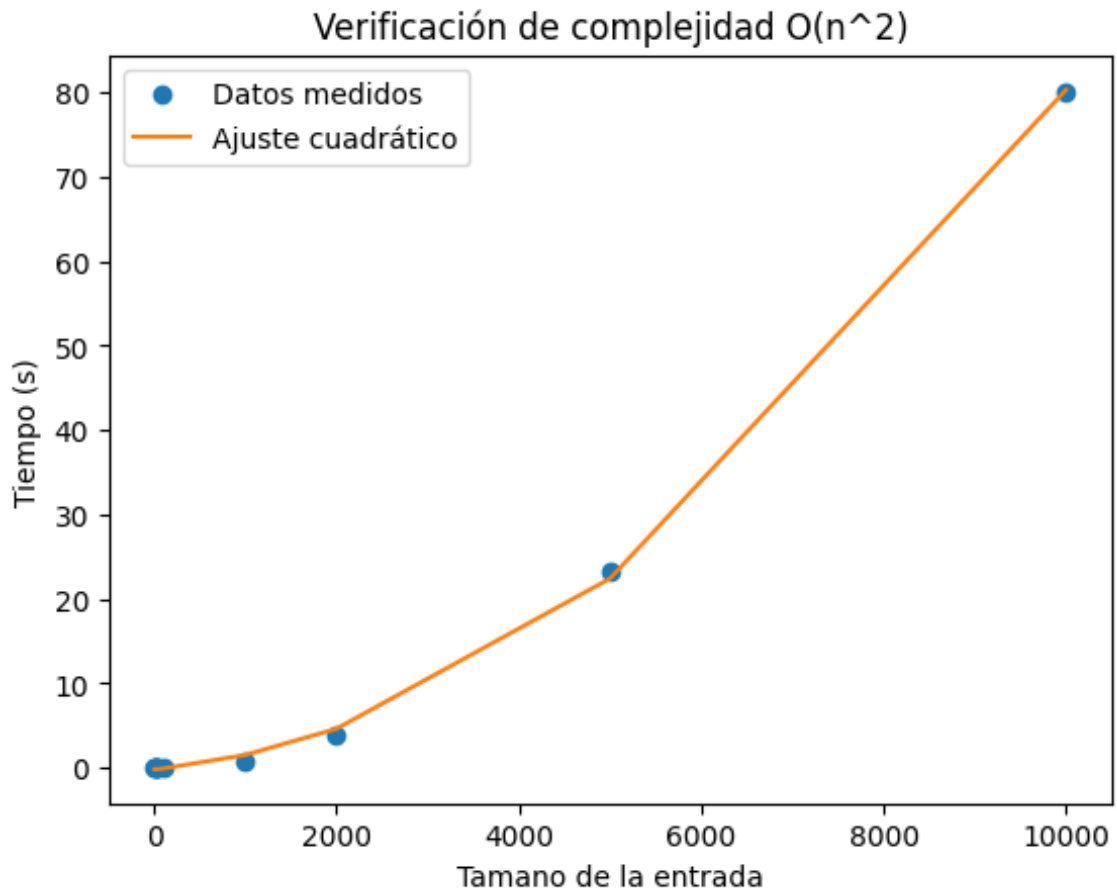
Para la reconstrucción de la solución se utilizó el siguiente algoritmo:

```
1 def reconstruir_solucion(monedas, dp):
2     i, j = 0, len(monedas) - 1
3     solucion = []
4
5     while i <= j:
6         if i == j:
7             solucion.append((monedas[i], "primera"))
8             break
9
10        turno_sophia = len(solucion) % 2 == 0
11
12        if turno_sophia:
13            # Sophia's optimal choice from dp
14            if monedas[i + 1] >= monedas[j]:
15                pick_i = monedas[i] + (dp[i + 2][j] if i + 2 <= j else 0)
16            else:
17                pick_i = monedas[i] + (dp[i + 1][j - 1] if i + 1 <= j - 1 else 0)
18
19            if monedas[i] >= monedas[j - 1]:
20                pick_j = monedas[j] + (dp[i + 1][j - 1] if i + 1 <= j - 1 else 0)
21            else:
22                pick_j = monedas[j] + (dp[i][j - 2] if i <= j - 2 else 0)
23
24            if pick_i >= pick_j:
25                solucion.append((monedas[i], "primera"))
26                i += 1
27            else:
28                solucion.append((monedas[j], "ultima"))
29                j -= 1
30        else:
31            # Mateo toma la moneda m s grande de los extremos
32            if monedas[i] >= monedas[j]:
33                solucion.append((monedas[i], "primera"))
34                i += 1
35            else:
36                solucion.append((monedas[j], "ultima"))
37                j -= 1
38
39    return solucion
```

El algoritmo hace uso de la ecuación de recurrencia para decidir, en cada paso, cuál es la moneda elegida, basándose en los valores calculados previamente en la tabla `dp`. Esta técnica garantiza que la solución reconstruida sea óptima, ya que se consideran todas las posibles elecciones y se elige la que iguala la ganancia en cada turno.

2.4. Mediciones

Para comprobar que la complejidad teórica es la correcta se realizó un ajuste cuadrático, utilizando la técnica de cuadrados mínimos. Dicho ajuste podemos verlo en la siguiente imagen:



Como podemos comprobar visualmente, las mediciones tienden a tener forma cuadrática, por lo que podemos concluir que las medidas tomadas corresponden con la complejidad teórica.

2.5. Conclusión

El algoritmo propuesto tiene complejidad $O(n^2)$, donde n es el número de monedas. Esta complejidad es independiente de los valores de las monedas, ya que la cantidad de operaciones realizadas depende únicamente del número de monedas, no de sus valores específicos. El algoritmo llena una matriz dp de tamaño $n \times n$, y cada subproblema se resuelve en $O(1)$, por lo que el número total de operaciones es proporcional a n^2 .

Cabe aclarar que, aunque los valores de las monedas afectan las decisiones que toma el algoritmo al seleccionar la moneda más beneficiosa en cada paso, esto no cambia el número de comparaciones y asignaciones realizadas. El algoritmo sigue un patrón de llenado de la matriz que es constante, independientemente de si las monedas tienen valores uniformes o muy dispares.

En conclusión, puede decirse que la variabilidad en los valores de las monedas no impacta significativamente los tiempos de ejecución del algoritmo. La eficiencia temporal del algoritmo se mantiene con complejidad $O(n^2)$, ya que, aunque los valores influyen las decisiones estratégicas, no alteran el número de operaciones necesarias para resolver el problema.

Tercera parte: Diversión NP-Completa

3. Nuevo juego: Batalla Naval Individual

En esta sección del informe, se analizará un algoritmo de backtranking para resolver un problema NP-Completo.

3.1. Problema

En primer lugar, se procederá a demostrar que el problema pertenece a la clase NP, describiendo el proceso para verificar una solución en tiempo polinómico. Para este propósito, se propone el siguiente algoritmo:

```
1 def es_batalla_navala(rec_fil, rec_col, matriz):
2     cont = 0
3     i=0
4     j=0
5     rec_col_finales = [0]*len(rec_col)
6
7     for fila in matriz:
8         for celda in fila:
9             if celda == 1:
10                 rec_col_finales[j] += 1
11                 cont += 1
12                 j += 1
13
14         if cont <= rec_fil[i]:
15             return False
16
17         i += 1
18         cont=0
19         j=0
20
21     for i in range(len(rec_col)):
22         if rec_col[i] < rec_col_finales[i]:
23             return False
24
25     return True
```

La función recibe como entrada las restricciones correspondientes a las filas y las columnas, así como una matriz que representa una posible solución, en la cual cada casilla con un valor de 1 indica la presencia de un barco.

El algoritmo realiza un recorrido completo de la matriz una única vez, donde cada iteración del bucle implica operaciones con complejidad $O(1)$. En consecuencia, la primera parte de la función tiene una complejidad de $O(n \times m)$.

Posteriormente, se itera sobre un vector auxiliar de longitud n , efectuando operaciones con complejidad $O(1)$ en cada iteración. Esto implica que esta etapa tiene una complejidad de $O(n)$.

Por lo tanto, al analizar ambas etapas, se concluye que la complejidad total del algoritmo puede acotarse como $O(n \times m)$. Dado que este algoritmo permite verificar una solución en tiempo polinómico, se puede determinar que el problema pertenece a la clase NP.

3.2. Reducción

Para comprobar que el problema de *batalla naval* es un problema NP-Completo, se eligió el problema *bin packing unario* para reducirlo a *batalla naval*. Cabe destacar que podemos probar que *bin packing unario* es un problema NP-Completo. Esto se puede ver trivialmente si reducimos el problema *b-partition*, donde el conjunto de números que queremos probar si suman el valor X en B subconjuntos lo podemos transformar a *bin packing unario* simplemente siendo B la cantidad de bins y X el valor C o cantidad.

Para reducir el problema de *bin packing unario* al problema de la *batalla naval*, primero debemos definir qué es cada elemento. El conjunto de n números en unario representará los barcos. La matriz se construirá de la siguiente manera: la cantidad de columnas será igual a la cantidad de contenedores o bins, y la cantidad de filas será la sumatoria de los números del conjunto más $n - 1$, siendo n la cantidad de números dentro del conjunto. Las restricciones de las columnas serán todas iguales al valor de la capacidad de los bins, y las restricciones de las filas representarán, en unario, los valores de los números del conjunto separados por 0.

Para entenderlo mejor, presentaremos un ejemplo: supongamos que tenemos el conjunto de números $\{11111, 11, 1111, 1\}$ y queremos ver si es posible que puedan dividirse en 11 bins de capacidad 111111 utilizando el problema de la batalla naval. Entonces transformamos los datos: los barcos serán $\{5, 2, 4, 1\}$, la matriz será de 2×15 , las restricciones de las columnas serán 6 y las restricciones de las filas siguen el siguiente patrón: las primeras 5 filas, siendo 5 el primer valor en el conjunto, tienen como restricción 1, luego la siguiente tiene como restricción 0. Las próximas 2 filas tendrán 1 como restricción y terminarán con un 0, y continuaremos hasta completar con todos los números del conjunto, finalizando en este caso con el 1. En el siguiente gráfico podemos ver como quedaría la matriz:

1		
1		
1		
1		
1		
0		
1		
1		
0		
1		
1		
1		
1		
0		
1		
-	6	6

Cuadro 1: Matriz transformada del Bin Packing al Batalla Naval

Ahora podemos ingresar nuestro nuevo set de datos al algoritmo que resuelva el problema de la batalla naval, lo que daría por resultado lo siguiente:

1	1	0
1	1	0
1	1	0
1	1	0
1	1	0
0	0	0
1	0	1
1	0	1
0	0	0
1	0	1
1	0	1
1	0	1
1	0	1
0	0	0
1	1	0
-	6	6

Cuadro 2: Matriz que representa la solución del problema de Batalla Naval

Podemos ver que en cada columna se almacenan, en unario, los valores que deberían pertenecer a cada uno de los bins.

Ahora que comprendemos la lógica de la reducción, procedamos a analizar si es posible transformar los datos del problema de bin packing en tiempo polinomial.

Primero, la transformación del conjunto de números al conjunto de barcos puede realizarse en tiempo constante, bajo la suposición de que ambos conjuntos están representados con enteros en lugar de en notación unaria. En el caso de notación unaria, la transformación implicaría una mayor complejidad, específicamente $O(n)$, donde n representa la cantidad de unos en la representación unaria.

A continuación, la definición de las restricciones para las columnas es directamente proporcional a la capacidad de los bins, lo cual puede realizarse en tiempo polinómico.

De manera análoga, la definición de las restricciones para las filas, aunque implique una lógica adicional, también es factible en tiempo polinómico. Cabe destacar que la complejidad asociada a este proceso sería $O(n)$, siendo n la cantidad de unos en la representación unaria.

Podemos concluir, entonces, que la reducción es válida, y el problema de la batalla naval se encuentra en NP-Completo.

3.3. ALGORITMO

3.3.1. Complejidad del Algoritmo

El número total de configuraciones posibles crece exponencialmente con la cantidad de barcos. Cada barco puede ocupar una posición específica dentro de la cuadrícula, considerando tanto su orientación como las restricciones del problema. Por ello, el espacio de búsqueda tiene un orden de $O(n^e)$, donde n representa el número total de celdas disponibles en la cuadrícula y e la cantidad de barcos.

Al ser un problema NP-completo, la solución óptima no puede ser encontrada en un tiempo polinomial en el peor de los casos. La complejidad del algoritmo sin optimización es exponencial, lo que hace que la resolución del problema en instancias grandes sea imposible debido al crecimiento desmesurado del espacio de soluciones.

3.3.2. Optimización mediante Podas

- Técnicas de Poda

Para hacer frente a la complejidad del problema, el algoritmo utiliza una serie de técnicas de poda que permiten reducir el espacio de búsqueda. A continuación, se describen las principales técnicas de poda implementadas en el algoritmo:

- Poda basada en la cantidad de barcos restantes

Una de las primeras podas que ocurre en el algoritmo es cuando no quedan más barcos para colocar. Esto sucede cuando el índice del barco alcanza el número total de barcos. En este caso, el algoritmo ha completado la colocación de todos los barcos, y no es necesario continuar explorando esa rama del árbol de búsqueda. La implementación de esta poda se encuentra en el siguiente bloque de código:

```
1 if barco_index >= len(self.barcos):  
2     self._update_best_solution(demanda_acumulada)  
3     return demanda_acumulada
```

- Poda por máxima demanda posible

Otra poda importante se basa en la demanda máxima posible de la configuración actual. Si la demanda máxima de la solución parcial es menor o igual a la demanda cumplida de la mejor solución encontrada hasta el momento, se puede descartar esa rama. Este tipo de poda es esencial para evitar explorar soluciones que no tienen el potencial de mejorar la mejor solución encontrada hasta ese momento. La poda por demanda máxima posible se implementa en el siguiente método:

```
1 def debe_podar(self, barco_index, demanda_acumulada):  
2     return (demanda_acumulada +  
3             sum(barco[0] for barco in self.barcos[barco_index:])) * 2 <=  
4             self.mejor_demanda_cumplida)
```

- Poda por demanda cumplida y barcos restantes

En esta poda, el algoritmo evalúa si la suma de la demanda cumplida hasta el momento y los barcos restantes es suficiente para superar la mejor solución encontrada. Si la combinación de estos dos elementos no es suficiente, la rama se poda. Esta poda se realiza en el siguiente fragmento de código:

```
1 def es_prometedora(self, fila, col, barco, orientacion, barco_index,  
2     demanda_acumulada):  
3     demanda_potencial = self.calcular_demanda_potencial(fila, col, barco,  
4     orientacion)  
5     return (demanda_acumulada + demanda_potencial +  
6             (sum(b[0] for b in self.barcos[barco_index+1:]) * 2) >  
7             self.mejor_demanda_cumplida)
```

- Poda cuando se alcanza el óptimo

Finalmente, cuando el algoritmo alcanza una solución óptima, es decir, cuando la demanda cumplida es igual o mayor a la demanda total, se poda la búsqueda. En este caso, no es necesario continuar explorando otras posibles soluciones. Esta poda se implementa en el siguiente bloque de código:

```
1 if mejor_demanda >= self.demanda_total:  
2     return mejor_demanda
```

3.4. Aproximación de John Jellicoe

3.4.1. Análisis del Algoritmo y de su Complejidad Temporal

1. Función verificar_adyacentes

```
1 def verificar_adyacentes(matriz, fila, columna, barco, es_vertical):
2     cantidad_filas = len(matriz)
3     cantidad_columnas = len(matriz[0])
4
5     if es_vertical:
6         inicio_fila = max(0, fila - 1)
7         fin_fila = min(cantidad_filas, fila + barco + 1)
8         inicio_columna = max(0, columna - 1)
9         fin_columna = min(cantidad_columnas, columna + 2)
10    else:
11        inicio_fila = max(0, fila - 1)
12        fin_fila = min(cantidad_filas, fila + 2)
13        inicio_columna = max(0, columna - 1)
14        fin_columna = min(cantidad_columnas, columna + barco + 1)
15
16    for i in range(inicio_fila, fin_fila):
17        for j in range(inicio_columna, fin_columna):
18            if i < cantidad_filas and j < cantidad_columnas and matriz[i][j]
19            == 1:
20                return False
21    return True
```

La complejidad temporal de esta función es $O(k)$, donde k es el área del rectángulo que se verifica alrededor del barco. En el peor caso: - Para un barco vertical: $k = (barco + 2) \times 3$ - Para un barco horizontal: $k = 3 \times (barco + 2)$

Como el tamaño del barco es una constante, podemos simplificar a $O(1)$.

2. Función colocar_vertical y colocar_horizontal

```
1 def colocar_vertical(matriz, fila, columna, barco, restriccion_columnas,
2     restriccion_filas):
3     if fila + barco > len(matriz):
4         return False
5
6     if restriccion_filas[columna] < barco:
7         return False
8
9     for i in range(fila, fila + barco):
10        if restriccion_columnas[i] <= 0:
11            return False
12
13    if verificar_adyacentes(matriz, fila, columna, barco, True):
14        for i in range(fila, fila + barco):
15            matriz[i][columna] = 1
16            restriccion_columnas[i] -= 1
17            restriccion_filas[columna] -= barco
18        return True
19    return False
20
21 def colocar_horizontal(matriz, fila, columna, barco, restriccion_columnas,
22     restriccion_filas):
23     if columna + barco > len(matriz[0]):
24         return False
25
26     if restriccion_columnas[fila] < barco:
27         return False
28
29     for i in range(columna, columna + barco):
30        if restriccion_filas[i] <= 0:
31            return False
```

La complejidad temporal se puede desglosar en:

- Verificación de límites: $O(1)$
- Verificación de restricciones: $O(\text{barco})$
- Llamada a verificar_adyacentes: $O(1)$
- Colocación del barco: $O(\text{barco})$

Complejidad total: $O(\text{barco})$, que se simplifica a $O(1)$ ya que el tamaño del barco es constante.

3.5. Función aproximacion_barcos

```
1 def aproximacion_barcos(matriz, restriccion_columnas, restriccion_filas,
2   barcos):
3     barcos = sorted(barcos, reverse=True)
4
5     while barcos:
6         barco_actual = barcos[0]
7         colocado = False
8
9         copia_restriccion_columnas = restriccion_columnas.copy()
10        copia_restriccion_filas = restriccion_filas.copy()
11
12        while max(max(copia_restriccion_columnas), max(copia_restriccion_filas)) > 0:
13            max_columna = max(copia_restriccion_columnas)
14            max_fila = max(copia_restriccion_filas)
15
16            if max_columna >= max_fila:
17                fila_max = copia_restriccion_columnas.index(max_columna)
18                for columna in range(len(matriz[0])):
19                    if colocar_horizontal(matriz, fila_max, columna,
20                                         barco_actual, restriccion_columnas, restriccion_filas):
21                        colocado = True
22                        break
23                copia_restriccion_columnas[fila_max] = 0
24            else:
25                columna_max = copia_restriccion_filas.index(max_fila)
26                for fila in range(len(matriz)):
27                    if colocar_vertical(matriz, fila, columna_max,
28                                       barco_actual, restriccion_columnas, restriccion_filas):
29                        colocado = True
30                        break
31                copia_restriccion_filas[columna_max] = 0
32
33            if colocado:
34                break
35
36        barcos.pop(0)
37
38    return matriz
```

Sea:

- a) n = número de filas de la matriz
- b) m = número de columnas de la matriz
- c) b = número de barcos

La complejidad temporal se puede desglosar en:

1. Ordenamiento inicial: $O(b \log b)$
2. Bucle principal para cada barco: $O(b)$ - Por cada barco:
 - Búsqueda del máximo en restricciones: $O(n + m)$
 - En el peor caso, intento de colocación en cada posición: $O(n \times m)$
 - Operaciones de colocación: $O(1)$

Complejidad total: $O(b \log b + b(n + m + nm))$

Simplificando, la complejidad dominante es $O(b \times n \times m)$

3.5.1. Cota Teórica

Para analizar que tan bueno es el algoritmo planteamos los siguientes tres ejemplos de prueba y calculamos $r(A)$:

■ Caso 1

Con los barcos $[2, 1, 1]$ y una matriz 3×3 con las siguientes características:

	2	3	1
2			
1			
2			

Cuadro 3: Matriz 3×3

Notese que el optimo sería:

	2	3	1
2	x	x	
1			
2	x		x

Cuadro 4: Matriz 3×3 optima

Y la aproximación:

	2	3	1
2		x	
1		x	
2			

Cuadro 5: Matriz 3×3 aproximada

En este caso $r(A) = 4/8 = 1/2$

■ Caso 2

Con los barcos $[1, 1, 1, 1]$ y una matriz 3×3 con las siguientes características:

	2	3	2
2		x	
1			
2			

Cuadro 6: Matriz 3×3

Notese que el optimo sería:

	2	3	2
2	x		x
1			
2	x		x

Cuadro 7: Matriz 3×3 optima

Y la aproximación:

	2	3	2
2		x	
1			
2	x		x

Cuadro 8: Matriz 3×3 aproximada

En este caso $r(A) = 6/8 = 3/4$

■ Caso 3

Con los barcos $[3, 3]$ y una matriz 3×4 con las siguientes características:

	3	4	3
1			
2			
2			
1			

Cuadro 9: Matriz 3×3

Notese que el optimo sería:

	3	4	3
1	x		
2	x		x
2	x		x
1			x

Cuadro 10: Matriz 3×3 optima

Y la aproximación:

	3	4	3
1		x	
2		x	
2		x	
1			

Cuadro 11: Matriz 3×3 aproximada

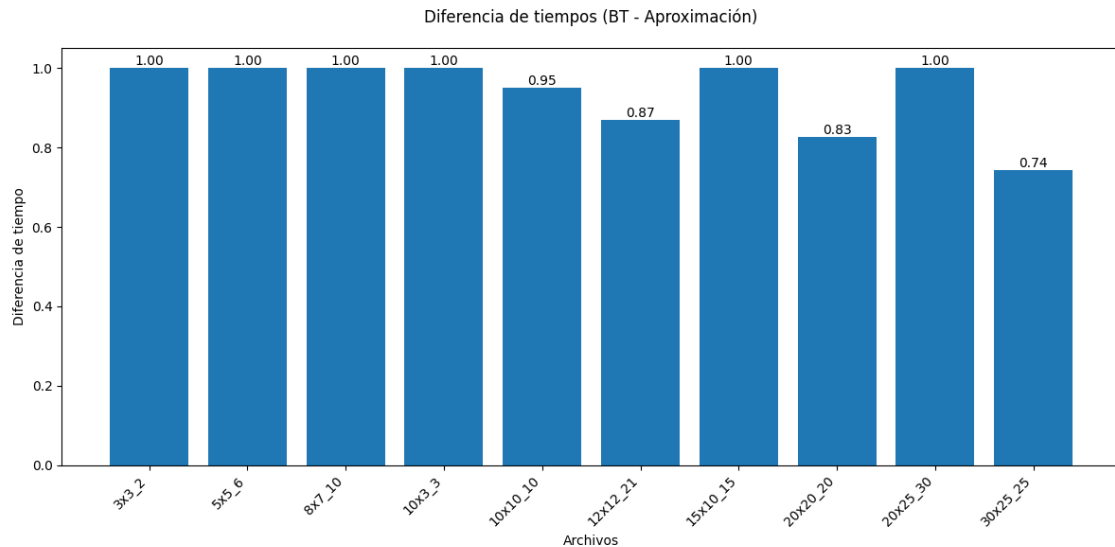
En este caso $r(A) = 6/12 = 1/2$

■ Conclusión Teórica

Se concluyó entonces, que $r(A) = \frac{1}{2}$ ya que es el peor de los ejemplos planteados.

3.5.2. Mediciones

Para comprobar la cota teórica realizamos las siguientes mediciones de resultados:



Notese que en los ejemplos probados la cota empírica es mejor que la teórica, siendo en el peor ejemplo 0.74.

Se puede concluir que la aproximación es significativamente precisa. A medida que el tamaño de los datos varía, la diferencia también cambia; sin embargo, esta siempre se mantiene por encima de 0,5 (como se comprobó teóricamente).

3.6. Conclusión

El algoritmo propuesto presenta una complejidad exponencial. Las técnicas de poda implementadas en la solución actual han sido seleccionadas para optimizar el rendimiento dentro de las posibilidades disponibles. Se considera que existen otras podas adicionales que podrían contribuir a mejorar aún más la eficiencia del algoritmo.